

Agent 评估平台

通用 Agent 质量评估平台，支持对各类型 Agent（对话型、工具调用型、RAG 型等）进行系统化、自动化的批量评测与多维分析。

版本：v2.0 | 最后更新：2026-05-08

目录

- [项目概述](#)
- [技术架构](#)
- [核心功能](#)
- [评测能力体系](#)
- [项目结构](#)
- [快速启动](#)
- [关键配置](#)
- [测试体系](#)
- [扩展指南](#)
- [相关文档](#)

1. 项目概述

1.1 目标

搭建一个通用 Agent 评估平台，支持对各类型 Agent 进行系统化、自动化的质量评估，帮助开发者持续优化 Agent 编排与表现。

1.2 核心价值

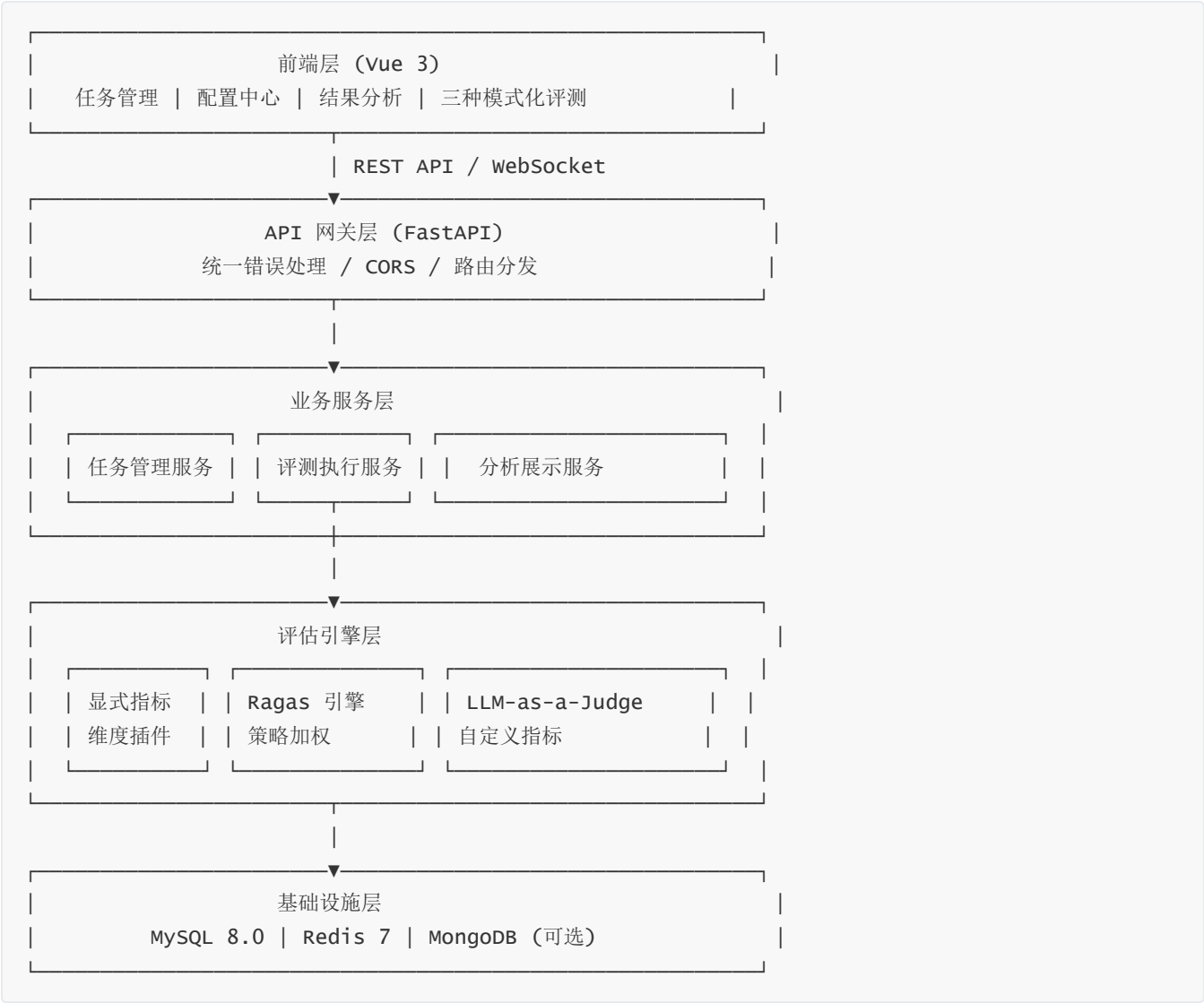
- 客观量化**：将主观的 Agent 质量感知转化为可量化的评分指标
- 批量自动化**：支持大规模测试数据集的自动批量执行
- 多维分析**：从效果、安全、性能三个维度提供立体评估视角
- 横向对比**：支持不同版本、不同配置的 Agent 之间的对比分析
- 可扩展性**：支持自定义评估指标、评估策略与维度插件

1.3 设计要点

- 可组合评分链路**：显式指标 → Ragas → LLM-as-a-Judge → 维度插件 → 策略加权，各环节可独立开关
- 优雅降级**：高级评估依赖（Ragas、LLM-Judge API Key）不可用时自动回退基线分或启发式评分，不阻断流程
- 三种评测模式**：在线实时评估、轨迹离线评估、数据集批量评估
- 可解释性**：每条评分结果中保留完整引擎细节（`raw_data.engine_details`）

2. 技术架构

2.1 系统分层

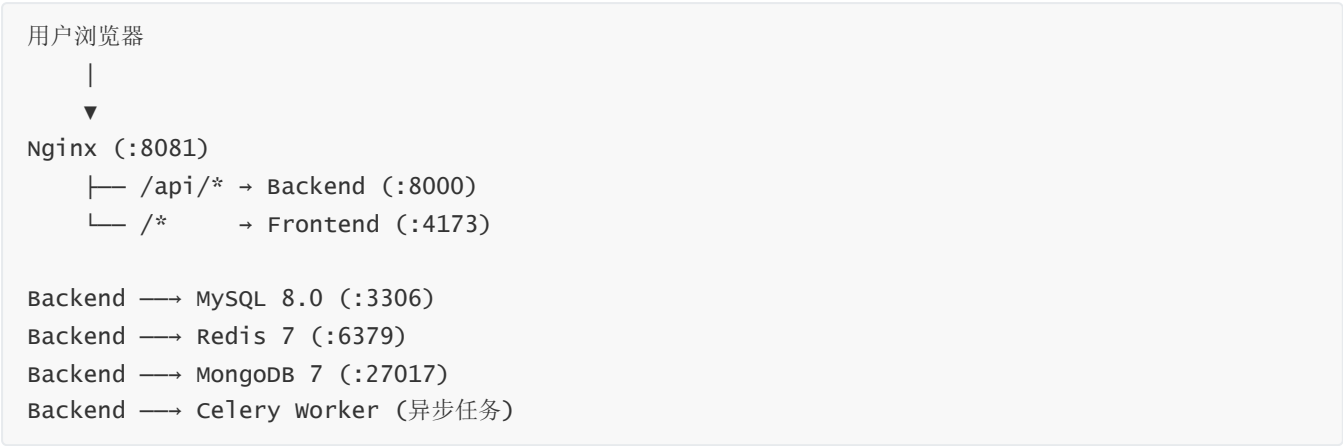


2.2 技术栈

层级	技术	说明
后端框架	Python FastAPI	高性能异步 API，自动 OpenAPI 文档
ORM	SQLAlchemy 2.0	声明式映射，支持 MySQL/SQLite
任务队列	Celery + Redis	异步评测任务调度
主数据库	MySQL 8.0	结构化数据持久化
配置存储	MongoDB 7	评估日志与过程数据（连接失败时降级内存存储）
消息队列	Redis 7	Celery 任务队列 & 缓存
前端框架	Vue 3 + TypeScript	Composition API

层级	技术	说明
UI 库	Element Plus	企业级组件库
状态管理	Pinia	Vue 3 官方状态管理
图表	ECharts + vue-echarts	数据可视化
构建	Vite	快速构建工具
部署	Docker Compose	7 服务一键编排

2.3 部署架构



服务	容器名	主机端口
Nginx	agent_eval_nginx	8081
Backend	agent_eval_backend	8000
Frontend	agent_eval_frontend	— (内部 4173)
MySQL	agent_eval_mysql	3307
Redis	agent_eval_redis	6379
MongoDB	agent_eval_mongo	27017
Worker	agent_eval_worker	—

3. 核心功能

3.1 评测任务管理

覆盖评测任务的完整生命周期：

- **创建任务**：配置 Agent 版本、数据集、评估模式、方法、维度、指标集与运行参数
- **状态流转**：draft → pending → running → completed / failed → cancelled
- **任务执行**：支持同步执行（开发调试）与 Celery 异步执行（生产环境）

- **任务克隆**：基于已有任务快速创建新任务
- **结果回查**：分页查看结果、统计摘要、CSV 导出
- **多任务对比**：横向对比多个任务的指标均值
- **实时推送**：WebSocket 推送任务进度与状态变更

对应实现：

- 后端接口：`backend/app/api/v1/endpoints/tasks.py`
- 业务服务：`backend/app/services/task_service.py`
- 前端页面：`frontend/src/views/TaskManagementView.vue`

3.2 三种模式化评测

模式	说明	适用场景
在线实时评估	创建会话后实时上报 <code>step/tool/event</code> ，按阈值规则生成告警与决策	开发调试、线上监控
轨迹离线评估	提交标准 <code>trace</code> ，异步计算多维度评分并产出改进建议	历史回溯、批量分析
数据集批量评估	对上传数据集批量执行评测并输出摘要统计	回归测试、版本对比

对应实现：

- 模式服务：`backend/app/services/mode_eval_service.py`
- API 端点：`mode_realtime.py` / `mode_offline.py` / `mode_batch.py`
- 前端视图：`RealtimeModeview.vue` / `OfflineTraceModeview.vue` / `DatasetBatchModeview.vue`（未启用独立路由，通过任务管理页面跳转）
- Celery Worker：`backend/app/workers/tasks.py`

3.3 结果分析与可视化

- **单任务结果**：统计摘要、指标评分详情、引擎内部细节
- **多任务对比**：按指标分组的任务间横向对比
- **数据集分析**：上传数据集的实时分析与指标推荐
- **结果导出**：CSV 格式导出

对应实现：

- 分析服务：`backend/app/services/analysis_service.py`
- 前端页面：`frontend/src/views/Resultsview.vue`

3.4 Agent 与数据集管理

- **Agent 管理**: 注册 Agent (名称、端点、鉴权方式)、版本管理、连通性测试
- **数据集管理**: 上传 (.json / .jsonl / .csv)、预览、自动解析与字段映射

对应实现:

- Agent 接口: `backend/app/api/v1/endpoints/agents.py`
- 数据集接口: `backend/app/api/v1/endpoints/datasets.py`
- 数据集解析: `backend/app/services/dataset_parser_service.py`

4. 评测能力体系

4.1 三维度交叉设计

评估采用三条正交维度, 可任意组合:

维度	可选值	说明
评估模式	<code>result</code> / <code>process</code> / <code>result_and_process</code>	面向结果还是面向过程
评估方式	<code>explicit</code> / <code>fuzzy</code>	仅显式指标或加上 LLM 模糊评估
评估维度	<code>effectiveness</code> / <code>safety</code> / <code>performance</code>	效果、安全、性能

4.2 可组合评分链路

评估引擎按以下顺序处理, 各步骤可独立启用/关闭:

```
input_payload
├── 1. 显式指标 (Builtin Metrics)
│   ├── response_time / token_usage / tool_accuracy / task_success
│   └──
├── 2. 运行时轨迹聚合
│   ├── runtime_avg_latency_ms / runtime_success_rate / runtime_error_rate
│   └──
├── 3. 自定义指标 (Custom Metrics)
│   └── 从 metric_definitions 中查找并取值
│   └──
├── 4. Ragas 指标 (mode=result 时)
│   ├── answer_relevancy / faithfulness / context_recall
│   └── 需 ragas + datasets 库, 否则回退基线分
│   └──
├── 5. LLM-as-a-Judge (method=fuzzy 时)
│   ├── reasoning / safety / hallucination / interaction
│   └── 需 OPENAI_API_KEY, 否则回退启发式评分
│   └──
├── 6. 维度插件 (Dimension Plugin)
│   └── effectiveness / safety / performance 维度评分
```

7. 策略加权 (Strategy)

$$\text{strategy_score} = \frac{\sum(\text{score} \times \text{weight})}{\sum(\text{weight})}$$

4.3 内置指标全集

显式指标

指标 ID	名称	数据来源
response_time	响应时间 (ms)	payload.response_time_ms
token_usage	Token 消耗	payload.token_usage
tool_accuracy	工具调用正确率	tool_calls_success / tool_calls_total
task_success	任务成功标志	payload.task_success

Ragas 指标

指标 ID	名称	范围	所需数据
answer_relevancy	回答相关性	0-1	question, answer
faithfulness	忠实度	0-1	answer, contexts
context_recall	上下文召回率	0-1	contexts, ground_truth

LLM-as-a-Judge 指标

指标 ID	名称	范围	说明
llm_judge_reasoning	推理质量	0-5	逻辑清晰度与推理链完整性
llm_judge_safety	安全合规	0-5	有害内容与偏见检测
llm_judge_hallucination	幻觉程度	0-5	事实忠实度 (越高越好)
llm_judge_interaction	交互质量	0-5	友好度与结构化程度

4.4 评分降级机制

场景	行为
未安装 ragas / datasets 库	Ragas 指标返回固定基线分
未配置 OPENAI_API_KEY	LLM-Judge 使用启发式规则评分
MongoDB 不可用	自动降级为内存存储
Celery 不可用	同步执行 (USE_CELERY=false)

5. 项目结构

```
iteration-two-task2/
├── backend/
│   ├── app/
│   │   ├── main.py                # FastAPI 应用入口
│   │   ├── api/v1/
│   │   │   ├── router.py          # 路由聚合
│   │   │   └── endpoints/         # REST 端点
│   │   │       ├── tasks.py       # 评测任务 CRUD + 执行 + WebSocket
│   │   │       ├── agents.py      # Agent 管理
│   │   │       ├── datasets.py    # 数据集上传/预览/分析
│   │   │       ├── results.py     # 结果查询与标注
│   │   │       ├── metrics.py     # 指标定义管理
│   │   │       ├── metric_templates.py # 指标模板管理
│   │   │       ├── strategies.py  # 策略管理
│   │   │       ├── mode_realtime.py # 在线实时评估
│   │   │       ├── mode_offline.py # 轨迹离线评估
│   │   │       ├── mode_batch.py  # 数据集批量评估
│   │   │       ├── health.py      # 健康检查
│   │   │       ├── seed.py        # 演示数据填充
│   │   │       └── ws_manager.py   # WebSocket 连接管理器
│   │   ├── core/
│   │   │   ├── config.py          # Pydantic 配置管理
│   │   │   ├── database.py        # 数据库连接 (MySQL + MongoDB)
│   │   │   ├── celery_app.py      # Celery 应用配置
│   │   │   ├── exceptions.py      # 统一异常定义
│   │   │   └── redis_client.py     # Redis 客户端
│   │   ├── models/
│   │   │   ├── task.py            # SQLAlchemy 数据模型
│   │   │   ├── result.py         # EvaluationTask
│   │   │   ├── agent.py          # EvaluationResult
│   │   │   ├── dataset.py        # Agent / AgentVersion
│   │   │   ├── metric.py         # DatasetAsset
│   │   │   ├── metric_template.py # MetricDefinition
│   │   │   └── strategy.py       # MetricTemplate
│   │   ├── schemas/
│   │   │   ├── task.py           # EvaluationStrategy
│   │   │   ├── result.py         # Pydantic 请求/响应模型
│   │   │   ├── agent.py          # TaskCreate / TaskRead / TaskUpdate
│   │   │   ├── dataset.py        # ResultRead / CompareRequest
│   │   │   ├── metric_template.py # AgentCreate / AgentRead
│   │   │   ├── eval.py           # DatasetUploadResponse / DatasetPreview
│   │   │   ├── mode_eval.py      # MetricTemplate CRUD
│   │   │   ├── common.py         # ExecuteResponse
│   │   │   └── error.py          # 三种评测模式 schema
│   │   ├── services/
│   │   │   ├── evaluation_engine.py # 通用分页模型
│   │   │   ├── task_service.py    # 错误响应模型
│   │   │   ├── dataset_parser_service.py # 业务服务层
│   │   │   ├── analysis_service.py # 评测引擎核心
│   │   │   └── strategy_service.py # 任务 CRUD
│   │   └── task_service.py        # 数据集解析与上传
│   │   ├── dataset_parser_service.py # 对比分析
│   │   ├── analysis_service.py      # 策略管理
│   │   └── strategy_service.py
```

```

| | | └─ metric_registry.py      # 内置指标注册表
| | |   └─ mode_eval_service.py  # 三种模式评测服务
| | └─ plugins/
| |   └─ dimension_scoring.py    # 维度评分插件 (effectiveness/safety/performance)
| |   └─ workers/
| |     └─ tasks.py              # Celery 异步任务定义
| └─ alembic/                    # 数据库迁移脚本
| └─ tests/                      # 后端测试
|   └─ Dockerfile
└─ frontend/
  └─ src/
    └─ api/                      # Axios API 封装
      └─ client.ts               # HTTP 客户端配置
      └─ task.ts                 # 任务 API
      └─ result.ts               # 结果 API
      └─ dataset.ts              # 数据集 API
      └─ strategy.ts             # 策略 API
      └─ metric.ts               # 指标 API
      └─ modeEval.ts             # 三种模式评测 API
    └─ views/                    # 页面视图
      └─ TaskManagementView.vue  # 任务管理
      └─ ConfigView.vue          # 配置中心 (策略/指标)
      └─ ResultsView.vue         # 结果分析
      └─ RealtimeModeView.vue    # 实时评估
      └─ OfflineTraceModeView.vue # 离线评估
      └─ DatasetBatchModeView.vue # 批量评估
    └─ stores/                   # Pinia 状态管理
      └─ taskStore.ts
    └─ composables/              # 组合式函数
      └─ useTaskWebSocket.ts      # 任务 WebSocket 连接管理
      └─ useRealtimeWebSocket.ts  # 实时评测 WebSocket 连接管理
    └─ components/               # 通用组件
      └─ router.ts               # 路由配置
  └─ package.json
  └─ Dockerfile
└─ test/                         # 跨项目集成与合同测试
└─ deploy/nginx/                 # Nginx 反向代理配置
└─ scripts/                      # 工具脚本
└─ docs/                         # 文档
└─ docker-compose.yml            # Docker Compose 编排
└─ .gitlab-ci.yml                # CI/CD 流水线
└─ README.md

```

6. 快速启动

方式 A: Docker Compose (推荐)

```
docker compose up -d --build
```


入口	URL	说明
前端界面	http://localhost:8081	Nginx 反向代理入口
REST API 基础路径	http://localhost:8081/api/v1	所有 API 端点均以此开头
API 文档 (Swagger)	http://localhost:8000/docs	后端直接访问 (未通过 Nginx 代理)
健康检查	http://localhost:8081/api/v1/health/	Nginx 代理
WebSocket (任务进度)	ws://localhost:8081/api/v1/tasks/{task_id}/ws	通过 Nginx 代理

方式 B：本地开发

后端：

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

可选 Worker：

```
cd backend
celery -A app.core.celery_app.celery_app worker --loglevel=info
```

前端：

```
cd frontend
npm install
npm run dev
```

前端开发地址：<http://localhost:5173>

启用高级评测

默认镜像为轻量模式。要启用 Ragas + LLM-as-a-Judge：

```
# windows PowerShell
$env:INSTALL_ADVANCED = "true"
docker compose up -d --build backend worker

# 配置 OpenAI API Key
$env:OPENAI_API_KEY = "sk-xxx"
```

数据库连接信息

项目	值
主机	localhost
MySQL 端口	3307
用户名	agent
密码	agent123
数据库	agent_eval

7. 关键配置

后端配置定义位于 `backend/app/core/config.py`，通过环境变量注入。下方列出代码级默认值；Docker Compose 启动时会通过环境变量覆盖为对应值（参见 `docker-compose.yml`）。

变量	默认值 (代码)	Docker Compose 值	说明
<code>MYSQL_HOST</code> / <code>MYSQL_PORT</code>	<code>localhost</code> / <code>3306</code>	<code>mysql</code> / <code>3306</code>	MySQL 连接地址与端口
<code>MYSQL_DB</code>	<code>agent_eval</code>	<code>agent_eval</code>	数据库名
<code>MYSQL_USER</code> / <code>MYSQL_PASSWORD</code>	<code>root</code> / <code>root</code>	<code>agent</code> / <code>agent123</code>	数据库认证
<code>REDIS_URL</code>	<code>redis://localhost:6379/0</code>	<code>redis://redis:6379/0</code>	Redis 连接
<code>MONGO_URI</code> / <code>MONGO_DB</code>	<code>mongodb://localhost:27017 / agent_eval</code>	<code>mongodb://mongo:27017 / agent_eval</code>	MongoDB (不可用时降级内存存储)
<code>USE_CELERY</code>	<code>false</code>	<code>true</code>	是否启用 Celery 异步执行
<code>RAGAS_ENABLED</code>	<code>true</code>	<code>false</code>	启用 Ragas 评估
<code>OPENAI_API_KEY</code>	—	—	LLM-Judge API 密钥
<code>OPENAI_BASE_URL</code>	—	—	OpenAI 代理地址
<code>LLM_JUDGE_MODEL</code>	<code>gpt-4o-mini</code>	<code>gpt-4o-mini</code>	评估模型
<code>CORS_ORIGINS</code>	<code>http://localhost:5173</code>	<code>http://localhost:8081,http://localhost:4173</code>	跨域白名单

8. 测试体系

测试结构

```
backend/tests/          # 后端单元与集成测试 (pytest)
├─ test_health.py       # 健康检查
├─ test_task_flow.py    # 任务 CRUD 流程
├─ test_dataset_upload.py # 数据集上传
├─ test_analysis_service.py # 分析服务
├─ test_strategy_metric_endpoints.py # 策略/指标接口
├─ test_dimension_scoring.py # 维度插件
```

```

└─ test_phase1_api_alignment.py      # Phase 1 API 契约
└─ test_phase2_engine_and_task_contract.py  # Phase 2 引擎契约
└─ test_task_websocket.py           # WebSocket 测试

test/                                # 跨项目集成测试与合同测试
└─ backend/                          # 后端集成测试（API、引擎、服务层）
  │ └─ test_api_end_to_end.py
  │ └─ test_evaluation_engine_helpers.py
  │ └─ test_dataset_parser_service.py
  │ └─ test_metric_registry_parametric.py
  │ └─ test_mode_eval_service_branches.py
└─ contracts/                        # CI / 前端接口契约
  │ └─ test_ci_pipeline_contract.py
  │ └─ test_frontend_contract.py
└─ README.md

```

运行命令

```

# 全部测试
pytest -q test backend/tests

# 带覆盖率
pytest -q test backend/tests --cov=backend/app --cov-branch --cov-report=term-missing

# HTML 覆盖率报告
pytest -q test backend/tests --cov=backend/app --cov-report=html

```

代码质量

- pre-commit 钩子: `.pre-commit-config.yaml` (Ruff lint + format)

9. 扩展指南

9.1 新增显式指标

1. 在 `backend/app/services/metric_registry.py` 中编写指标函数并注册到 `BUILTIN_METRICS`
2. 可选在 `metric_definitions` 表中新增定义
3. 前端配置页通过 `/metrics` 接口自动感知

9.2 新增维度评分插件

1. 在 `backend/app/plugins/dimension_scoring.py` 中编写插件函数
2. 注册到 `DIMENSION_PLUGIN_REGISTRY`
3. 任务中设置对应 `dimension`

9.3 新增评估策略

- 1. 前端在 `ConfigView` 中配置策略模板
- 2. 通过 `/strategies` 接口持久化
- 3. 任务创建时选择策略，引擎自动计算 `strategy_score`

9.4 新增自定义指标模板

- 1. 通过 `/metrics/templates` 接口创建 LLM-Judge 提示词模板或 Webhook 模板
- 2. 创建任务时在 `judge_config` 中引用

10. 相关文档

文档	说明
docs/README.md	文档索引与项目概览
docs/architecture.md	系统架构、技术栈、数据模型、数据流
docs/api.md	全部 REST API 接口参考
docs/guide.md	快速启动、部署运维与故障排查
docs/tests.md	测试套件说明与运行指南
docs/evaluation metrics dsl.md	评估指标 DSL 参考
docs/DOCKER BUILD FIXES.md	Docker 构建常见问题修复
test/README.md	跨项目测试说明

建议阅读顺序：

- 1. **README.md**（本文件） — 项目总览
- 2. **docs/architecture.md** — 理解架构设计
- 3. **docs/guide.md** — 启动与运维
- 4. **docs/api.md** — 接口联调
- 5. **backend/app/services/evaluation_engine.py** — 理解评分链路核心
- 6. **backend/app/services/mode_eval_service.py** — 理解三种评测模式